Session 03 - Menu Loop and In-Memory Documents - Tests

Python Contoso Vault - Session 03 of 07

Part 1 - Core: vault.py

Goal

Wrap the cipher in a menu that keeps running until the user quits. Let the user create documents, list them and read them back. Keep each body encrypted in memory, and give a reader three attempts to type the right key.

Run `python vault.py` in the terminal. Type each input line and press Enter after it. To try one function, open the Python shell in the same folder and run `from vault import check_key, create_document`.

Provided

Copy your own `caesar.py` from your Session 02 folder into this folder. It defines `encrypt(text: str, key: int) -> str` and `decrypt(text: str, key: int) -> str`.

Instructions

The class builds steps 1, 2 and 4, leaving out the two key checks under step 4.2.3.3 and all of step 4.2.5, and imports only `encrypt`. When the class build stops, choice `3` prints `Invalid choice.` and the key at Create has no checks. Finish at home: import `decrypt`, then add step 3, the two key checks under step 4.2.3.3 and step 4.2.5.

1. Import `encrypt` and `decrypt` from `caesar.py`.
2. Define a function `create_document` that takes `documents` as a list of dictionaries, `title` as a string, `body` as a string and `key` as an integer.
   2.1. Give the new document a unique ID, as an integer. IDs count up from 1 in the order the documents are created, so the third document has ID 3.
   2.2. Encrypt `body` with `key`.
   2.3. Encrypt `CONTOSO-VAULT` with `key`, as the check value.
   2.4. Build `document` as a dictionary in the shape of this example, where the values of `body` and `check` are encrypted:

```
{
    "id": 1,
    "title": "Payroll",
    "body": "Vdodulhv jr rxw rq 28 Qryhpehu.",
    "check": "FRQWRVR-YDXOW",
}
```

   2.5. Add `document` to the end of `documents`.
   2.6. Return `document` as a dictionary.
3. Define a function `check_key` that takes `document` as a dictionary and `key` as an integer.
   3.1. Decrypt the check value of `document` with `key`.
   3.2. Return `True` if the result is `CONTOSO-VAULT`, and `False` otherwise.
4. Run these steps only when the file runs as a program:
   4.1. Start `documents` as an empty list.
   4.2. Repeat until the user quits:
      4.2.1. Print the menu as these four lines:

```
1. Create
2. List
3. Read
4. Quit
```

      4.2.2. Read `choice` as a string, with the prompt `"Choice: "` on the same line.
      4.2.3. On `1`:

4.2.3.1. Read `title` as a string, with the prompt `"Title: "` on the same line.
4.2.3.2. Read `body` as a string, with the prompt `"Body: "` on the same line.
4.2.3.3. Read `key` as an integer, with the prompt `"Key: "` on the same line.
4.2.3.3.1. If the input is not a whole number, print `Please enter a whole number.` and read `key` again with the prompt `"Key: "`.
4.2.3.3.2. If `key` is a whole number outside 1 to 25, print `Key must be from 1 to 25.` and read `key` again with the prompt `"Key: "`.
4.2.3.4. Create `document` from `documents`, `title`, `body` and `key`.
4.2.3.5. Print `Saved document N.`, where `N` is the ID of `document`.
4.2.4. On `2`:
4.2.4.1. Print one line `ID. Title` per document, in ID order.
4.2.4.2. Print `No documents.` if there are none and go back to the menu.
4.2.5. On `3`:
4.2.5.1. Read `document_id` as an integer, with the prompt `"ID: "` on the same line.
4.2.5.1.1. If the input is not a whole number, print `Please enter a whole number.` and read `document_id` again with the prompt `"ID: "`.
4.2.5.2. Search `documents` for the document with the ID `document_id`.
4.2.5.3. Print `No such document.` and go back to the menu if the search finds no document.
4.2.5.4. Otherwise, give the user three attempts at the key. For each attempt:
4.2.5.4.1. Read `key` as an integer, with the prompt `"Key: "` on the same line. Any whole number uses an attempt, including a number outside 1 to 25.
4.2.5.4.1.1. If the input is not a whole number, print `Please enter a whole number.` and read `key` again with the prompt `"Key: "`. This does not use an attempt.
4.2.5.4.2. If `key` is right, print the decrypted body alone on one line, with no label, and go back to the menu.
4.2.5.4.3. Print `Wrong key.` if `key` is wrong.
4.2.5.5. Print `Too many attempts.` and go back to the menu after the third wrong key.
4.2.6. On `4`, print `Goodbye.` and end the program.
4.2.7. On any other choice, print `Invalid choice.`

Constraints

- Each prompt is its label, a colon and one trailing space, as in `input("Title: ")`. Type each prompt exactly as shown. A program with different prompt text does not meet the test plan.
- The program prints the menu and reads each input in the order the steps give.
- A key saved at Create is always from 1 to 25.
- Two keys that differ by 26 open the same document.
- `create_document`, `check_key` and importing the file neither print nor read input.
- The program never stops with a Python error message, whatever the user types. It ends only when the user chooses `"4"`.

Design

The following rules say how to write the code. Follow them.

- Decide whether a guessed key is right with `check_key`.
- Keep only the encrypted body in memory. Never store the decrypted body.
- Write the whole-number input check once, in one function.
- Give every variable and function a type hint. `document` has the type hint `dict`, and `documents` has the type hint `list[dict]`.

Input validation

Required for every key and ID, as the Instructions set out. Assume every title and body uses English letters, digits, spaces and punctuation only. An empty title or body is valid.

Test plan

Function tests

`documents` starts as an empty list. `document_1` is the result of test 1.

Test	Call	Expected output	Actual output	Pass
1	`create_document(documents, "Payroll", "Salaries go out on 28 November.", 3)`	`{"id": 1, "title": "Payroll", "body": "Vdodulhv jr rxw rq 28 Qryhpehu.", "check": "FRQWRVR-YDXOW"}`. `documents` holds it		
2	Then `create_document(documents, "Kandy rent", "Kandy office rent is Rs 150,000.", 11)`	`{"id": 2, "title": "Kandy rent", "body": "Vlyoj zqqtnp cpye td Cd 150,000.", "check": "NZYEZDZ-GLFWE"}`. `documents` holds 2		
3	`create_document([], "Empty", "", 5)`	`{"id": 1, "title": "Empty", "body": "", "check": "HTSYTXT-AFZQY"}`		
4	`check_key(document_1, 3)`	`True`		
5	`check_key(document_1, 4)`	`False`		
6	`check_key(document_1, 29)`	`True`		
7	`check_key(document_1, -23)`	`True`		
8	`check_key(document_1, 0)`	`False`		

Program tests

`python vault.py` reads the input lines in order. "Create Payroll" stands for the four input lines `"1"`, `"Payroll"`, `"Salaries go out on 28 November."`, `"3"`. Every run prints the four menu lines and the prompt `"Choice: "` before each choice, and ends with the `Goodbye.` line. The expected output cells leave out the menu lines, the prompts and the `Goodbye.` line. They list the remaining output lines, such as `Saved document 1.`

Test	Input lines	Expected output	Actual output	Pass
9	`4`	Only the menu and `Goodbye.`		
10	`2`, `4`	`No documents.`		
11	Create Payroll, `2`, `4`	`Saved document 1.`, `1. Payroll`		
12	Create Payroll, `3`, `1`, `3`, `4`	`Saved document 1.`, `Salaries go out on 28 November.`		
13	Create Payroll, `3`, `1`, `5`, `3`, `4`	`Saved document 1.`, `Wrong key.`, `Salaries go out on 28 November.`		
14	Create Payroll, `3`, `1`, `5`, `6`, `7`, `2`, `4`	`Saved document 1.`, `Wrong key.` three times, `Too many attempts.`, `1. Payroll`		
15	Create Payroll, `3`, `1`, `abc`, `5`, `6`, `3`, `4`	`Saved document 1.`, `Please enter a whole number.`, `Wrong key.`, `Wrong key.`, `Salaries go out on 28 November.`		
16	`1`, `Payroll`, `Salaries go out on 28 November.`, `abc`, `0`, `26`, `3`, `4`	`Please enter a whole number.`, `Key must be from 1 to 25.`, `Key must be from 1 to 25.`, `Saved document 1.`		
17	`3`, `1`, `4`	`No such document.`		
18	Create Payroll, `3`, `two`, `0`, `4`	`Saved document 1.`, `Please enter a whole number.`, `No such document.`		
19	`7`, `x`, empty line, `4`	`Invalid choice.` three times		
20	Create Payroll, `1`, `Kandy rent`, `Kandy office rent is Rs 150,000.`, `11`, `2`, `3`, `2`, `11`, `4`	`Saved document 1.`, `Saved document 2.`, `1. Payroll`, `2. Kandy rent`, `Kandy office rent is Rs 150,000.`		
21	`import vault`	Reads no input and prints nothing		

In every program test, the program ends after `Goodbye.` and shows no Python error message.

Done criteria

- [ ] Every test plan row gives exactly the expected result.
- [ ] The script runs from the terminal with `python vault.py`.

Part 2 - Challenge: leak.py

Goal

Show why a stored check value is a weakness. Recover a document's key from its check value alone, without trying any keys.

Keep `caesar.py` and `vault.py` in the same folder as `leak.py`. Open the Python shell there, run `from leak import recover_key`, then call `recover_key` with each document named in the test plan.

Provided

`vault.py` defines `create_document(documents, title, body, key) -> dict` and `check_key(document, key) -> bool`. `caesar.py` defines `decrypt(text, key) -> str`. Import them only if your code uses them.

Instructions

1. Define a function `recover_key` that takes `document` as a dictionary in the shape of this example, where the values of `body` and `check` are encrypted:

```
{
    "id": 1,
    "title": "Payroll",
    "body": "Vdodulhv jr rxw rq 28 Qryhpehu.",
    "check": "FRQWRVR-YDXOW",
}
```

   1.1. Work out `key` from the check value of `document`, which is `CONTOSO-VAULT` encrypted with `key`. `key` is an integer from 1 to 25. Call `decrypt` and `check_key` at most once per document.
   1.2. Return `key` as an integer.

Constraints

- The returned key opens the document: `check_key(document, recover_key(document))` is `True`.
- `recover_key` and importing the file neither print nor read input.

Design

The following rules say how to write the code. Follow them.

- Work out the key without trying keys.
- Give every variable and function a type hint. `document` has the type hint `dict`.

Input validation

Not required. Assume every document comes from `create_document` with a key from 1 to 25.

Test plan

Test	Document	Expected output	Actual output	Pass
1	Check value `FRQWRVR-YDXOW`	`3`		
2	Check value `NZYEZDZ-GLFWE`	`11`		
3	Check value `DPOUPTP-WBVMU`	`1`		
4	Check value `BNMSNRN-UZTKS`	`25`		
5	Check value `PBAGBFB-INHYG`	`13`		
6	30 documents from `create_document`, keys 1 to 25	Each returns its own key		

Done criteria

☐ Every test plan row returns exactly the expected key.

Marking

This take-home is marked out of 100, with a pass mark of 60.

Part	Tests	Design	Total
Core	64	16	80
Challenge	16	4	20
Total	**80**	**20**	**100**

Guidelines

- Test marks show that the program works. The instructor's tests run it on every row of the test plan, plus further cases.
- Design marks show that the code is built well. The instructor reads the code against each Design rule.
- A pass needs 60 marks from the Core alone. Challenge marks count towards the total once the Core reaches 60.
- Marking uses the last commit pushed by 11:59 PM on the day before the next session.
- The submission is one GitHub link to the fork, posted on the portal.